STOCKPILOT AI

Smart Inventory & Demand Intelligence System

PYTHON PROGRAMMING PROJECT REPORT

SUBMITTED BY
SWARUP CHOWDHURY
ROLL NUMBER
26MCI10229
SECTION
26MAM4(B)

in partial fulfillment for the award of the degree of

Master of Computer Applications in Artificial Intelligence and Machine Learning (MCA-AIML)

Subject: Python Programming



Submitted to:
Mr. Sanjeev Kumar (E20624)

CHANDIGARH UNIVERSITY
1st Year • 1st Semester • October 2026

INDEX

StockPilot AI — Python Programming Project Report

This index follows the requested project-report format. Page numbers refer to the generated PDF version.

No.	Content	Page
1	Project Title and Introduction	4
2	Problem Statement and Objectives	5
3	Tools and Technologies Used	6
4	Algorithm / Methodology	8
5	Implementation	10
6	Output / Results (Screenshots)	12
7	Conclusion	15
A	Appendix: Project File Map and References	16

Project at a glance

StockPilot AI is a Python Flask inventory application that combines persistent database operations, role-based access, explainable restocking analytics, transaction-safe stock updates, and a responsive dashboard for small businesses.

Student: Swarup Chowdhury | Course: MCA (AI/ML) | Section: 26MAM4(B) | Roll No.: 26MCI10229

PROJECT INFORMATION

Academic and system profile

Student information	Details
Name	Swarup Chowdhury
Course	MCA (AI/ML)
College / University	Chandigarh University
Section	26MAM4(B)
Roll number	26MCI10229
Year / Semester	1st Year / 1st Semester
Subject	Python Programming
Submitted to	Mr. Sanjeev Kumar (E20624)

Project synopsis

StockPilot AI is a database-backed inventory and demand-intelligence web application. It is designed for grocery shops, supermarkets, retail stores, wholesalers, and small businesses that need a reliable way to understand stock health and decide what to purchase next.

The Python layer is responsible for routing, authentication, session handling, validation, business services, analytics, report generation, and database access. The system is intentionally implemented as a working application instead of a static dashboard: sales reduce inventory, purchase receipts increase inventory, waste uses FEFO batch selection, and reports are generated from stored records.

Important scope note

The Preview uses a managed MySQL-compatible database when DATABASE_URL is available and keeps SQLite as a clearly separated local fallback. Oracle SQL and PL/SQL artifacts are included as the academic production target; this report focuses on the Python application layer.

1. Project Title and Introduction

1.1 Project title

StockPilot AI — Smart Inventory & Demand Intelligence System

1.2 Introduction

Stock management is a daily operational activity, but many small businesses still rely on notebooks or disconnected spreadsheets. StockPilot AI centralizes product records, suppliers, purchase orders, sales, stock batches, expiry information, alerts, audit history, and explainable recommendations in one application.

The application answers three practical questions: What is currently in stock? What needs attention today? What should be purchased next? A dashboard gives an executive view, while Inventory, Sales & Billing, Purchasing, Restocking AI, Expiry & Waste, Analytics, Algorithm Lab, Reports, and Business Assistant pages support detailed work.

1.3 Project scope

- Secure login, logout, session handling, roles, CSRF checks, and account-scoped workspaces.
- Product management with SKU, barcode, category, supplier, price, quantity, reorder level, and expiry fields.
- Transactional sales, Decimal-safe totals, stock validation, FEFO deduction, audit movement, and printable/report-ready records.
- Purchase orders, receipts, suppliers, low-stock signals, expiry and waste workflows, and CSV exports.
- Explainable restocking calculations, baseline forecasting, algorithm demonstrations, and a bounded business assistant.

2. Problem Statement and Objectives

2.1 Problem statement

Manual inventory methods make it difficult to maintain accurate quantities, identify low-stock products, monitor expiry batches, reconcile sales with stock, compare suppliers, and justify purchase decisions. They also increase the risk of duplicate SKUs, incorrect stock deductions, unrecorded waste, and weak auditability.

The project solves this problem through a centralized Python web application connected to a normalized database. Every important operation is validated at the backend, written as a transaction, and reflected in the product quantity, stock batch, movement ledger, audit history, and dashboard metrics.

2.2 Objectives

1. Develop a working Python Flask inventory application with a responsive user interface.
2. Create a persistent data layer for products, suppliers, sales, purchase orders, batches, movements, notifications, users, and audit logs.
3. Maintain transaction safety so a failed sale, receipt, or waste action does not leave partial data behind.
4. Implement role-based access for Admin / Owner, Inventory Manager, and Staff users.
5. Use explainable demand and restocking formulas instead of presenting an unverified black-box prediction.
6. Demonstrate custom algorithms that have direct business value in inventory search, prioritization, routing, and FEFO handling.
7. Provide useful reports, onboarding guidance, database persistence, and an application suitable for academic demonstration.

Success criterion

A user should be able to log in, inspect stock health, record a sale, receive a purchase order, review a restocking explanation, record waste, and download a report without manually editing database rows.

3. Tools and Technologies Used

Layer	Technology / tool	Purpose
Programming language	Python 3.11	Backend logic, validation, analytics, algorithms, tests, and report/export endpoints.
Web framework	Flask 3.0.3	Routes, sessions, JSON APIs, role checks, request handling, and application shell.
Database	Managed MySQL-compatible DB	Persistent Preview/production records shared across application restarts.
Local fallback	SQLite	Clearly separated local development adapter for offline evaluation.
Oracle target	Oracle Database + PL/SQL	Academic production schema and package artifacts for core operations.
Frontend	HTML5, CSS3, JavaScript	Responsive dashboard, forms, charts, tables, modals, and notifications.
Analytics	Python Decimal and standard-library calculations	Currency-safe totals, demand history, valuation, trends, and restocking signals.
Security	Hashed passwords, secure sessions, CSRF tokens, role checks	Protect accounts and state-changing operations.
Testing	PyTest and Python compilation	Validate algorithms, formulas, error handling, and core workflows.
Deployment	Docker and managed hosting	Package and publish the Flask application with environment-driven configuration.
Version control	Git	Checkpointed project history and reproducible delivery.

3.1 Important Python project files

File	Responsibility
app.py	Flask application, login/registration, protected routes, API responses, report downloads, and health endpoint.
db.py	Database adapters, schema creation, managed database selection, tenancy migration, and demo seeding.
services.py	Product creation, sales, purchase receipts, waste, reports, dashboard data, and assistant intents.
analytics.py	Demand calculations, restocking formulas, category mix,

	daily sales, movers, and baseline forecast.
algorithms.py	Custom Merge Sort, Binary Search, hash lookup, Dijkstra, priority queue, and FEFO.
templates/index.html	Server-rendered application shell and login / new-admin forms.
static/app.js and styles.css	Client-side view rendering, API actions, charts, forms, responsive styling, and toasts.

4. Algorithm / Methodology

4.1 Layered methodology

8. Authenticate the user and create a protected session containing role and workspace scope.
9. Read current products, sales, suppliers, batches, purchase orders, and movements from the database.
10. Validate a business operation in Flask and the service layer before changing any records.
11. Apply the operation atomically: update product quantity, batch quantity, movement ledger, and audit history together.
12. Generate dashboard metrics, notifications, recommendations, reports, and assistant answers from stored data.

4.2 Explainable restocking method

The restocking engine is a transparent business-rule baseline. It is not presented as a trained machine-learning model. The formulas make each recommendation inspectable by a student, manager, or examiner.

Calculation	Formula / explanation
Average daily demand	Units sold during the lookback period ÷ lookback days.
Lead-time demand	Average daily demand × supplier lead time in days.
Safety stock	A conservative buffer based on recent demand; fallback uses reorder-level information.
Reorder point	Lead-time demand + safety stock.
Target stock	The larger of a minimum-stock buffer or reorder point plus a planning horizon.
Suggested quantity	max(0, target stock − current stock).
Confidence	High, Medium, or Fallback according to available sales history.

4.3 Algorithms demonstrated

Algorithm	Business use	Complexity
Merge Sort	Stable ordering of products, sales, or recommendations.	O(n log n) time; O(n) space
Binary Search	Exact SKU lookup after enforcing sorted input.	O(log n); sorted precondition
Hash-based lookup	Fast SKU or barcode resolution during checkout.	O(1) average; O(n) space

Dijkstra	Shortest route distance for nonnegative warehouse edges.	O((V+E) log V)
Priority Queue	Surface critical restocking alerts first.	O(n log n) build-and-pop
FEFO	Consume the batch with the earliest expiry first.	O(n log n) sorting

4.4 FEFO example

If a product has batches expiring on 05-Nov, 12-Nov, and 25-Nov, a sale or waste operation first selects the 05-Nov batch. If that batch is insufficient, the remaining quantity is taken from 12-Nov, then 25-Nov. This reduces expiry risk and makes the deduction explainable.

5. Implementation

5.1 Database and persistence

The Python application uses an environment-driven adapter. When DATABASE_URL is supplied, the managed MySQL-compatible database is used and data survives process restarts and published deployments. SQLite remains a local fallback. Workspace IDs scope users and operational data so a new admin receives a private empty workspace while the fictional demo dataset remains confined to the demo account.

The data model contains users, workspaces, categories, suppliers, products, stock batches, sales, sale items, purchase orders, purchase order items, stock movements, notifications, audit logs, and inventory policies. Foreign-key relationships, unique identifiers, status fields, and validation rules keep the records consistent.

5.2 Python route and service flow

```
@app.post("/api/sales")
@roles("admin", "manager", "staff")
def add_sale():
    try:
        return jsonify(process_sale(
            db(), request.get_json(silent=True) or {},
            session["user"]["id"], workspace_id()
        )), 201
    except Exception as exc:
        return jsonify({"error": str(exc)}), 400
```

The route applies the role and CSRF boundary, then delegates the actual business operation to process_sale. The service validates stock, calculates Decimal-safe totals, selects FEFO batches, updates quantities, writes movement and audit rows, and commits only after all lines succeed. Any exception triggers rollback.

5.3 Account onboarding and isolation

13. The user selects Create new admin on the login page.
14. The backend validates name, workspace name, unique email, and password length.
15. A new workspace and owner user are inserted into the managed database.
16. The user is signed in and sees an empty training dashboard instead of demo records.
17. All later reads and writes use the workspace ID stored in the protected session.

5.4 Security and validation

- Passwords are stored as hashes rather than plain text.
- Protected routes require an authenticated session cookie.
- State-changing routes require the session CSRF token.
- Roles are enforced on the backend, not only by hiding interface controls.
- Duplicate SKU and barcode values are rejected by validation and database constraints.
- The assistant accepts supported parameterized intents and does not execute arbitrary SQL or code.

5.5 Testing and deployment

The project was checked with Python compilation and automated tests covering sorting, binary search, graph validation, alert priority, FEFO selection, restocking calculations, Decimal totals, and insufficient-stock rejection. The application is packaged with Docker and configured to honor the PORT environment variable. The production dependency manifest includes Flask, PyMySQL, pandas, NumPy, oracledb, PyTest, and python-dotenv.

6. Output / Results (Screenshots)

6.1 Login and new-admin registration output

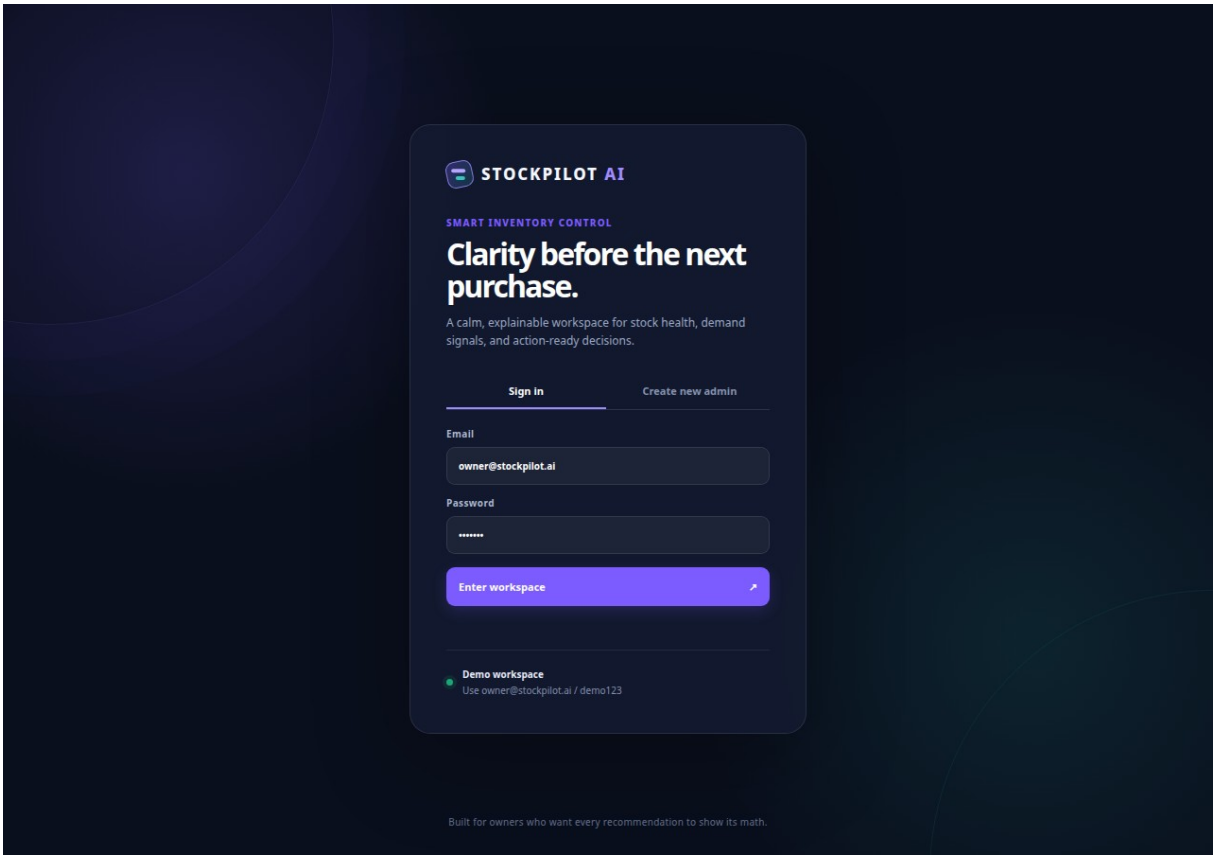


Figure 1. StockPilot AI login screen with Sign in and Create new admin options.

The login screen presents the demo account for evaluation and a separate new-admin path. The registration flow creates a private workspace rather than copying the demo inventory. This output demonstrates that the Python backend and client-side interface are connected to the account model.

6.2 Executive dashboard output

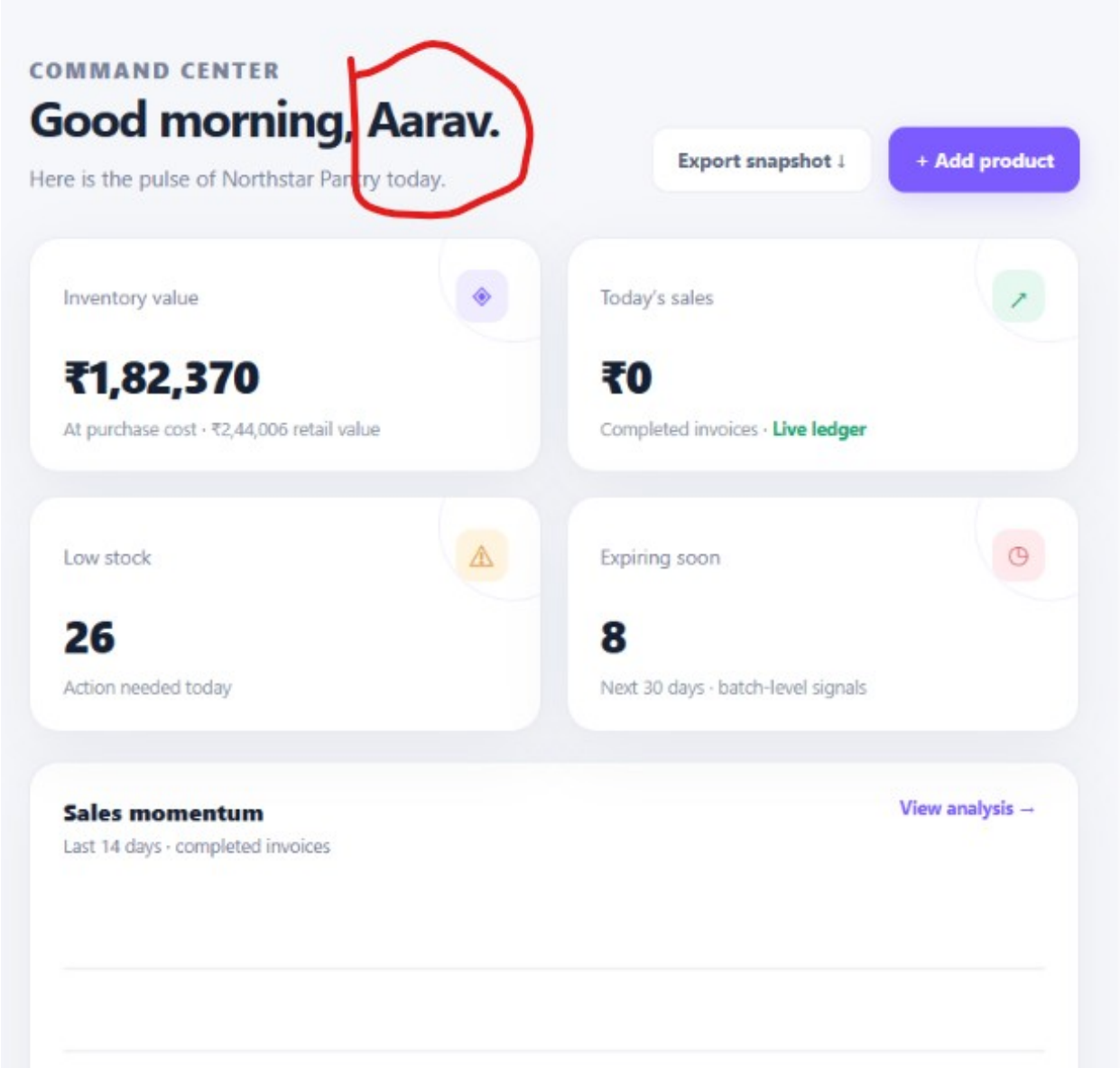


Figure 2. StockPilot AI executive dashboard using database-backed demo records.

The dashboard summarizes inventory value, retail value, today's sales, low-stock count, expiry signals, sales momentum, category mix, critical recommendations, and recent movement. In the captured demo snapshot, the displayed metrics include an inventory value of ₹1,82,370, low stock of 26 products, and 8 expiring signals. These values are presented as fictional demonstration data and are calculated from stored records.

6.3 Functional results

Tested result	Observed behavior
Demo login	owner@stockpilot.ai / demo123 opens the Northstar Pantry demo workspace.
New-admin registration	Creates a private workspace and starts with no operational

	demo records.
Workspace isolation	A record created in a new workspace is not visible to the demo account.
Product creation	Creates a product, opening batch, stock movement, and audit record.
Sale processing	Validates stock, calculates totals, deducts FEFO batches, and commits as one operation.
Purchase receipt	Increases stock, creates receipt movement, and updates purchase status.
Reports	Returns inventory, low-stock, sales, expiry, and audit data as CSV-ready records.
Automated tests	Four core tests pass after Python compilation.

7. Conclusion

StockPilot AI demonstrates how Python can be used to build a complete, practical, database-backed business application. The project goes beyond form display: it maintains a stock ledger, protects operations with authentication and roles, preserves data through a managed database, and exposes explainable inventory decisions through analytics and algorithms.

The system is useful for an academic Python Programming demonstration because it combines core Python programming concepts with real application concerns: functions and modules, exception handling, Decimal arithmetic, data structures, sorting and searching, graph algorithms, priority queues, database transactions, HTTP routes, validation, report generation, and automated tests.

The project can be extended with barcode scanning, supplier price history, richer forecasting models, scheduled notifications, Oracle integration tests, multilingual support, and mobile-friendly receiving workflows. The existing baseline deliberately communicates forecast limitations and keeps business recommendations reviewable by a human user.

Final outcome

The delivered application provides a working foundation for small-business inventory intelligence: every recommendation shows its math, every stock-affecting action is auditable, and every new account receives its own persistent workspace.

APPENDIX: PROJECT FILE MAP AND REFERENCES

A.1 Project file map

Area	Files / artifacts
Python backend	app.py, config.py, db.py, services.py, analytics.py, algorithms.py
Frontend	templates/index.html, static/app.js, static/styles.css, static/favicon.svg
Testing	tests/test_core.py
Database artifacts	oracle/schema.sql, oracle/inventory_pkg.sql, oracle/seed_data.sql
Documentation	docs/ARCHITECTURE.md, docs/DAA.md, docs/ORACLE_SETUP.md, docs/TRAINING_GUIDE.md, docs/MULTI_USER_PERSISTENCE.md
Deployment	requirements.txt, Dockerfile, .env.example, public/manus-routes.json

A.2 References used for this report

- StockPilot AI project source code and database artifacts, Chandigarh University academic project workspace.
- StockPilot AI User Training Guide, project documentation.
- StockPilot AI Architecture and Algorithm Insights documentation.
- Captured StockPilot AI application screenshots from the project Preview and supplied dashboard image.

Prepared for: Python Programming | Chandigarh University | 1st Year, 1st Semester | October 2026